

Time Forge: Um Sistema para Geração Automática de Grades Horárias em Instituições de Ensino Usando uma Abordagem Híbrida com Constraint Satisfaction Problem, Backtracking e First Fit Decreasing

Enzo B. de Oliveira¹, Ezequiel H. G. Muller¹, Lucas Blanger¹

¹Universidade Regional Integrada do Alto Uruguai e das Missões Departamento de Engenharias e Ciência da Computação Caixa Postal 743 – 99.709-910 – Erechim – RS – Brasil

112963@aluno.uricer.edu.br, 111624@aluno.uricer.edu.br, 110978@aluno.uricer.edu.br

1 INTRODUÇÃO

A elaboração de grades horárias em instituições de ensino constitui um problema de otimização combinatória de reconhecida complexidade computacional. Classificado formalmente na literatura como um problema NP-difícil (GAREY; JOHNSON, 1979), o processo de construção de horários acadêmicos envolve a alocação simultânea de múltiplos recursos: professores, salas, turmas e intervalos de tempo, além de um conjunto denso de restrições rígidas e flexíveis cuja complexidade cresce exponencialmente com o tamanho da instituição. A prevalência de processos manuais de elaboração de horários em instituições de ensino e sua susceptibilidade a erros são amplamente reconhecidas na literatura (SCHAERF, 1999), contexto no qual o cenário brasileiro não é exceção.

O problema da geração automática de horários acadêmicos, internacionalmente denominado *University Course Timetabling Problem* (UCTP), tem sido objeto de extensa investigação científica ao longo das últimas décadas. Wren (1996) situa historicamente os primeiros esforços computacionais para sua resolução ainda nos anos 1960, período em que as limitações de hardware impunham abordagens heurísticas altamente simplificadas. A formalização do problema como instância de *Constraint Satisfaction Problem* (CSP), em uma abordagem na qual variáveis precisam receber valores de seus respectivos domínios de forma a satisfazer um conjunto de restrições (RUSSELL; NORVIG, 2020), o que permitiu o desenvolvimento de métodos sistemáticos e mais robustos para sua resolução.

Dentre as estratégias algorítmicas consagradas para a resolução de CSPs, o *Backtracking Search* destaca-se pela sua completude teórica: o algoritmo explora exaustivamente o espaço de soluções, desfazendo atribuições inconsistentes e retrocedendo na árvore de decisão até encontrar uma solução válida ou determinar a inviabilidade do problema (DECHTER, 2003). Apesar de sua completude, a eficiência prática do *Backtracking* puro degrada-se rapidamente para instâncias de maior porte, o que motivou a integração de heurísticas de ordenação e propagação de restrições, tais como *Minimum Remaining Values*

(MRV) e *Forward Checking* (HARALICK; ELLIOT, 1980). No contexto de alocação de recursos físicos, a heurística *First Fit Decreasing* (FFD), originalmente proposta para o problema de Bin Packing (JOHNSON et al., 1974), demonstrou eficácia na ordenação de entidades por demanda decrescente, reduzindo a ocorrência de estados sem solução ao final do processo de alocação.

A relevância acadêmica e prática do problema é atestada pela vasta literatura existente. Bonutti et al. (2012) e Di Gaspero et al. (2007), no contexto das competições ITC (International Timetabling Competition), propuseram benchmarks amplamente utilizados para a avaliação comparativa de abordagens ao UCTP. Soluções baseadas em metaheurísticas, como Algoritmos Genéticos (COLORNI; DORIGO; MANIEZZO, 1991), *Simulated Annealing* (KIRKPATRICK; GELATT; VECCHI, 1983) e Tabu Search (GLOVER, 1986), foram amplamente exploradas, evidenciando o trade-off entre qualidade da solução e custo computacional.

Contudo, abordagens híbridas que combinam a sistematicidade do CSP com *Backtracking* e heurísticas determinísticas apresentam vantagens práticas em cenários institucionais reais, onde a rastreabilidade das decisões e a interpretabilidade do processo são requisitos fundamentais (MÜLLER; MURRAY, 2010).

No contexto brasileiro, a automação da geração de horários permanece pouco explorada no âmbito de soluções integradas e de código aberto voltadas a instituições de médio porte. A ausência de sistemas acessíveis que conciliam robustez algorítmica, persistência relacional e interfaces de gestão amigáveis representa uma lacuna tecnológica relevante. Diante desse cenário, o presente trabalho propõe o desenvolvimento do TimeForge, um sistema para geração automática de grades horárias acadêmicas fundamentado em uma abordagem híbrida que combina modelagem via CSP, busca por *Backtracking* com estratégias de retrocesso inteligente e a heurística FFD para alocação de recursos físicos. O sistema é implementado sobre uma arquitetura moderna composta por *backend* em Java com Spring Boot, interface web desenvolvida em Vue.js e banco de dados relacional PostgreSQL, permitindo persistência estruturada das entidades acadêmicas e geração de grades completas ou parciais, sendo estas últimas entregues como resultado de melhor esforço em cenários de inviabilidade total, aliviando o trabalho manual dos coordenadores acadêmicos. A Figura 1 apresenta a identidade visual do sistema TimeForge, desenvolvida para representar o projeto.

Figura 1 — Logo do sistema TimeForge



Fonte: Elaborado pelos autores (2026)

Este artigo está organizado da seguinte forma: a Seção 2 apresenta os objetivos geral e específicos do trabalho; a Seção 3 expõe a fundamentação teórica dos algoritmos, modelos e tecnologias adotados; a Seção 4 descreve a metodologia de desenvolvimento do sistema; a Seção 5 apresenta e discute os resultados obtidos; e a Seção 6 traz as considerações finais.

2 OBJETIVOS

2.1 Objetivo Geral

Desenvolver o TimeForge, um sistema computacional para geração automática de grades horárias acadêmicas, fundamentado em uma abordagem híbrida que combina modelagem por *Constraint Satisfaction Problem* (CSP), busca por *Backtracking* e heurística *First Fit Decreasing* (FFD), capaz de produzir alocações completas ou parciais respeitando as restrições operacionais de instituições de ensino.

2.2 Objetivos Específicos

- Modelar o domínio acadêmico professores, turmas, disciplinas, salas e slots de horário, em um esquema relacional persistido em PostgreSQL, refletindo fielmente as restrições e entidades do ambiente institucional real;

- Implementar o núcleo algorítmico de geração de horários com base na combinação de CSP, *Backtracking* com retrocesso inteligente e heurística FFD para ordenação e alocação de recursos físicos por capacidade decrescente;
- Desenvolver uma API RESTful em Java com Spring Boot para expor os serviços de cadastro, configuração e geração de grades, garantindo separação de responsabilidades e extensibilidade arquitetural;
- Construir uma interface web em Vue.js voltada à usabilidade dos coordenadores acadêmicos, permitindo o gerenciamento das entidades do sistema e a visualização das grades geradas de forma clara e intuitiva;
- Elaborar a documentação técnica da API RESTful desenvolvida, descrevendo os endpoints disponíveis, os modelos de dados, os parâmetros de requisição e os retornos esperados, de forma a facilitar a manutenção e a integração futura do sistema;
- Avaliar o desempenho e a escalabilidade do sistema mediante testes com volumes crescentes de dados, identificando os limites operacionais do algoritmo em termos de tempo de processamento e capacidade de alocação.

3 FUNDAMENTAÇÃO TEÓRICA

Esta seção apresenta os conceitos e tecnologias que fundamentam o desenvolvimento do TimeForge, organizados em dois eixos principais: os fundamentos algorítmicos que sustentam a geração automática de horários e os fundamentos de Engenharia de *Software* que orientam a construção e a organização do sistema.

3.1 O Problema de Geração de Horários Acadêmicos

3.1.1 University Course Timetabling Problem (UCTP)

O *University Course Timetabling Problem* (UCTP) consiste em atribuir disciplinas, professores e turmas a slots de tempo e espaços físicos, satisfazendo simultaneamente um conjunto de restrições obrigatórias e, quando possível, restrições de preferência (SCHAERF, 1999). Formalmente, o problema é classificado como NP-difícil, o que indica que não há algoritmos conhecidos capazes de resolvê-lo de forma eficiente para todos os casos, especialmente em instâncias de maior complexidade (GAREY; JOHNSON, 1979).

A literatura distingue dois tipos de restrições no UCTP. As restrições rígidas (*hard constraints*) são inegociáveis e sua violação torna a solução inválida, como a proibição de um professor ministrar duas aulas simultaneamente ou a sobreposição de turmas em um mesmo

horário. As restrições flexíveis (*soft constraints*), por sua vez, representam preferências institucionais ou docentes cuja satisfação é desejável, mas não obrigatória (BONUTTI et al., 2012). No TimeForge, o foco recai sobre a satisfação completa das restrições rígidas, com as restrições de disponibilidade dos professores tratadas como restrições obrigatórias do modelo.

O interesse científico pelo UCTP é atestado pela realização das competições *International Timetabling Competition* (ITC), cujas edições de 2002, 2007 e 2019 consolidaram *benchmarks* e metodologias de avaliação amplamente adotados pela comunidade (DI GASPERO et al., 2007). Diversas abordagens foram propostas ao longo dos anos, incluindo métodos exatos baseados em programação inteira, metaheurísticas como Algoritmos Genéticos (COLORNI; DORIGO; MANIEZZO, 1991) e *Simulated Annealing* (KIRKPATRICK; GELATT; VECCHI, 1983), e métodos baseados em satisfação de restrições, que fundamentam a abordagem adotada neste trabalho

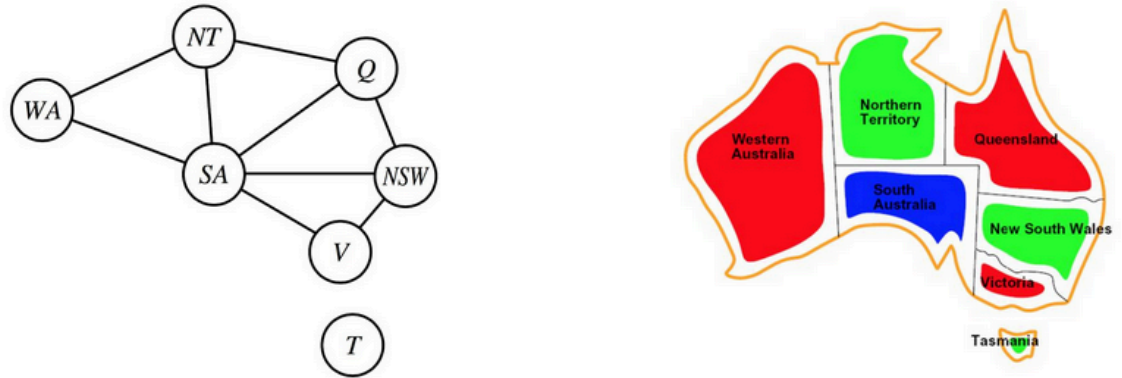
3.2 Fundamentos Algorítmicos

3.2.1 Constraint Satisfaction Problem (CSP)

Um Problema de Satisfação de Restrições (*Constraint Satisfaction Problem* — CSP) é definido formalmente por uma tripla (X, D, C) , onde $X = \{x_1, x_2, \dots, x_n\}$ representa o conjunto de variáveis do problema, $D = \{d_1, d_2, \dots, d_n\}$ representa os domínios de valores possíveis para cada variável, e C representa o conjunto de restrições que limitam as combinações válidas de valores (RUSSELL; NORVIG, 2020). Uma solução para um CSP é uma atribuição completa de valores às variáveis tal que todas as restrições em C sejam satisfeitas simultaneamente.

No contexto do TimeForge, cada aula a ser agendada constitui uma variável do CSP. O domínio de cada variável é o produto cartesiano entre os slots de horário disponíveis e as salas com capacidade suficiente para a turma em questão. As restrições do sistema proíbem atribuições que resultem em conflitos de professor, turma ou sala em um mesmo slot de horário, além de exigir que o horário alocado esteja dentro da disponibilidade declarada do docente responsável.

Figura 2 — Representação do problema de coloração de mapas da Austrália: (a) grafo de restrições correspondente e (b) mapa com as regiões e uma solução possível



Fonte: RUSSELL; NORVIG (2020).

A Figura acima apresenta um exemplo clássico utilizado na literatura para ilustrar CSPs: o problema de coloração do mapa da Austrália. Na representação (b), cada região geográfica (como Western Australia — WA, Northern Territory — NT, South Australia — SA, entre outras) corresponde a uma variável do problema. Já na representação (a), o mesmo problema é modelado como um grafo de restrições, no qual cada nó representa uma variável e cada aresta indica a existência de uma restrição entre duas regiões adjacentes.

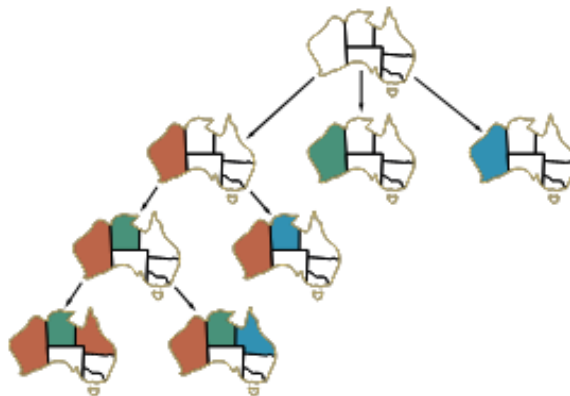
Nesse contexto, o domínio de cada variável consiste em um conjunto finito de cores possíveis, enquanto as restrições impõem que regiões vizinhas não podem receber a mesma cor. Essa dupla representação (geográfica e em grafo) evidencia como problemas do mundo real podem ser abstraídos em estruturas formais, facilitando a aplicação de algoritmos de busca e técnicas de propagação de restrições.

A escolha do CSP como modelo de resolução é justificada pela sua expressividade e pelo suporte a técnicas sistemáticas de busca e propagação de restrições. Dechter (2003) demonstra que a modelagem via CSP permite a aplicação de algoritmos de consistência, como a consistência de arco (AC-3) para reduzir os domínios das variáveis antes e durante a busca, diminuindo significativamente o espaço de soluções a ser explorado.

3.2.2 Backtracking Search

O algoritmo de *Backtracking Search* é o método de busca sistemática mais utilizado para a resolução de CSPs (RUSSELL; NORVIG, 2020). Sua lógica consiste em atribuir valores às variáveis sequencialmente, verificando a cada passo se a atribuição parcial atual é consistente com as restrições do problema. Quando uma inconsistência é detectada, o algoritmo desfaz a última atribuição realizada (operação denominada *backtrack*) e tenta o próximo valor disponível no domínio da variável, retrocedendo na árvore de decisão até encontrar uma atribuição válida ou determinar que o problema não possui solução (DECHTER, 2003).

Figura 3 — Ilustração do processo de busca por backtracking no problema de coloração de mapas, evidenciando a exploração de diferentes atribuições de valores



Fonte: RUSSELL; NORVIG (2020).

A Figura acima ilustra o processo de exploração do espaço de busca pelo algoritmo de backtracking no contexto do problema de coloração de mapas. Cada configuração apresentada representa uma atribuição parcial ou completa de cores às regiões da Austrália. À medida que o algoritmo avança, diferentes combinações são testadas, sendo descartadas aquelas que violam as restrições de adjacência e mantidas aquelas que permanecem consistentes. Esse processo pode ser interpretado como a exploração de uma árvore de busca, na qual cada ramo corresponde a uma possível sequência de atribuições.

A eficiência prática do *Backtracking* puro pode ser severamente comprometida em instâncias de maior porte devido à explosão combinatória do espaço de busca. Para mitigar esse problema, a literatura propõe heurísticas de ordenação de variáveis e valores. A heurística *Minimum Remaining Values* (MRV), também denominada *fail-first*, prioriza a atribuição de variáveis com menor número de valores disponíveis em seus domínios,

antecipando a detecção de falhas e podando ramos inviáveis da árvore de busca mais cedo (HARALICK; ELLIOT, 1980).

No TimeForge, a ordenação das aulas a serem alocadas considera a carga horária semanal e a especificidade das restrições de cada TurmaDisciplina, priorizando os casos mais restritos.

3.2.3 Heurística First Fit Decreasing (FFD)

A heurística *First Fit Decreasing* (FFD) é originalmente associada ao problema de *Bin Packing*, que consiste em empacotar itens de tamanhos variados no menor número possível de contêineres de capacidade fixa (JOHNSON et al., 1974). O princípio central da FFD é ordenar os itens em ordem decrescente de tamanho antes de iniciar o processo de alocação, garantindo que os itens maiores (e portanto mais difíceis de acomodar) sejam alocados primeiro, enquanto os espaços residuais menores são preenchidos pelos itens remanescentes.

No contexto do TimeForge, a heurística FFD é aplicada à alocação de salas. As turmas são ordenadas por capacidade decrescente, e as salas de maior capacidade são reservadas prioritariamente às turmas com maior número de alunos. Essa estratégia reduz a probabilidade de que turmas grandes encontrem apenas salas insuficientes ao final do processo de alocação, um problema frequente em abordagens ingênuas de atribuição sequencial. Johnson et al. (1974) demonstraram que o FFD produz soluções com razão de aproximação garantida em relação ao ótimo, o que confere respaldo teórico à sua adoção como componente heurístico do algoritmo.

3.3 Engenharia de Software

3.3.1 Engenharia de Requisitos

A Engenharia de Requisitos compreende o conjunto de atividades destinadas a identificar, documentar, analisar e validar as necessidades dos stakeholders de um sistema de software (SOMMERVILLE, 2011). Segundo Pressman e Maxim (2016), a qualidade dos requisitos levantados na fase inicial do projeto tem impacto direto na qualidade do produto final, uma vez que requisitos ambíguos ou incompletos são uma das principais causas de retrabalho e falha em projetos de software.

No desenvolvimento do TimeForge, a engenharia de requisitos orientou a identificação das restrições funcionais do algoritmo, como as regras de não sobreposição de professores, turmas e salas, e dos requisitos não funcionais relacionados ao desempenho e à

usabilidade da interface. Essa etapa subsidiou a definição do modelo de dados e a especificação dos *endpoints* da API, garantindo rastreabilidade entre as necessidades institucionais e as decisões de implementação.

3.3.2 Programação Orientada a Objetos

A Programação Orientada a Objetos (POO) é um paradigma de desenvolvimento de *software* que organiza o sistema em torno de objetos, entidades que encapsulam estado (atributos) e comportamento (métodos), em contraposição à organização centrada em procedimentos e funções (BOOCH, 1994).

Os quatro pilares fundamentais do paradigma são o encapsulamento, que restringe o acesso direto ao estado interno dos objetos; a herança, que permite o reaproveitamento de comportamentos entre classes relacionadas; o polimorfismo, que possibilita que objetos de tipos distintos respondam de forma uniforme a uma mesma interface; e a abstração, que modela apenas os aspectos relevantes do domínio, ocultando detalhes de implementação (RUMBAUGH et al., 1991).

No desenvolvimento do TimeForge, o paradigma orientado a objetos é aplicado tanto na modelagem das entidades do domínio acadêmico — Professor, Turma, Disciplina — quanto na estruturação do algoritmo de geração de horários, onde cada componente do CSP é representado como um objeto com responsabilidades bem definidas. A adoção da POO em conjunto com o Spring Boot favorece a organização do código em camadas coesas, a testabilidade das unidades e a manutenibilidade do sistema ao longo do seu ciclo de vida.

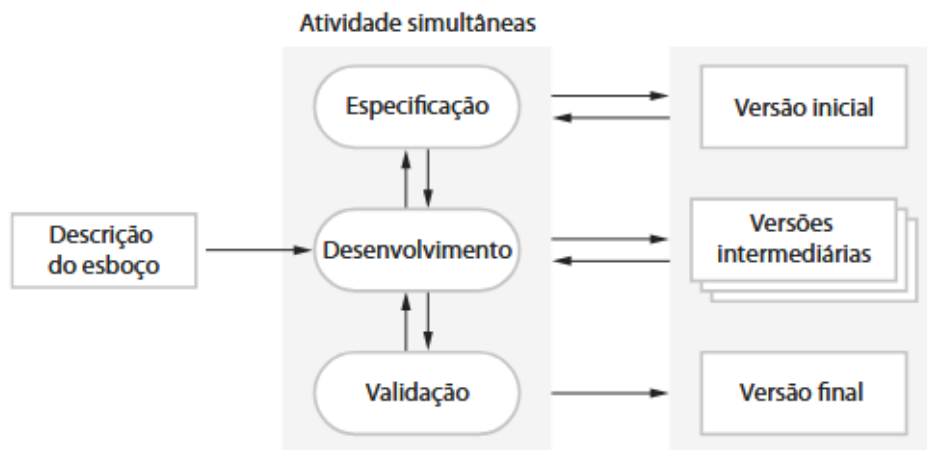
3.3.3 Processo de Desenvolvimento Iterativo e Incremental

O desenvolvimento iterativo e incremental é uma abordagem de engenharia de software na qual o sistema é construído em ciclos sucessivos, cada um produzindo uma versão mais completa e refinada do produto (SOMMERVILLE, 2011). Diferentemente do modelo cascata, que exige a conclusão integral de cada fase antes do início da seguinte, o modelo iterativo admite o surgimento e a incorporação de novos requisitos ao longo do desenvolvimento, respondendo de forma natural à evolução do entendimento do problema por parte da equipe.

Conforme ilustrado na Figura 4, o processo iterativo organiza o desenvolvimento em ciclos que percorrem as atividades de especificação, desenvolvimento, validação e implantação de forma repetida, com cada iteração incorporando refinamentos e novos

incrementos ao sistema (SOMMERVILLE, 2011). Essa estrutura cíclica permite que a equipe responda a mudanças de requisitos sem comprometer a integridade do que já foi construído.

Figura 4 — Modelo de Desenvolvimento Iterativo e Incremental



Fonte: Sommerville (2011)

No desenvolvimento do TimeForge, esse processo se manifestou nas seguintes etapas com retroalimentação contínua: levantamento inicial dos requisitos e funcionalidades do sistema; modelagem do banco de dados e elaboração do Diagrama de Classes; implementação incremental dos componentes; testes contínuos a cada incremento desenvolvido; e validação do comportamento do sistema frente aos requisitos estabelecidos. À medida que a implementação avançava, novos requisitos e refinamentos foram identificados e incorporados ao escopo, caracterizando a natureza iterativa do processo adotado.

3.3.4 Testes de Software

Os testes de software constituem uma atividade essencial para a garantia da qualidade de sistemas computacionais, sendo responsáveis pela verificação do comportamento do sistema frente aos requisitos especificados e pela identificação de defeitos antes da entrega (MYERS; SANDLER; BADGETT, 2011). A literatura classifica os testes em diferentes níveis: testes unitários, que verificam unidades isoladas de código; testes de integração, que avaliam a interação entre componentes; e testes de sistema, que validam o comportamento do software como um todo (PRESSMAN; MAXIM, 2016).

No contexto do TimeForge, os testes assumem um papel especialmente relevante na validação do algoritmo de geração de horários, uma vez que a correção das alocações

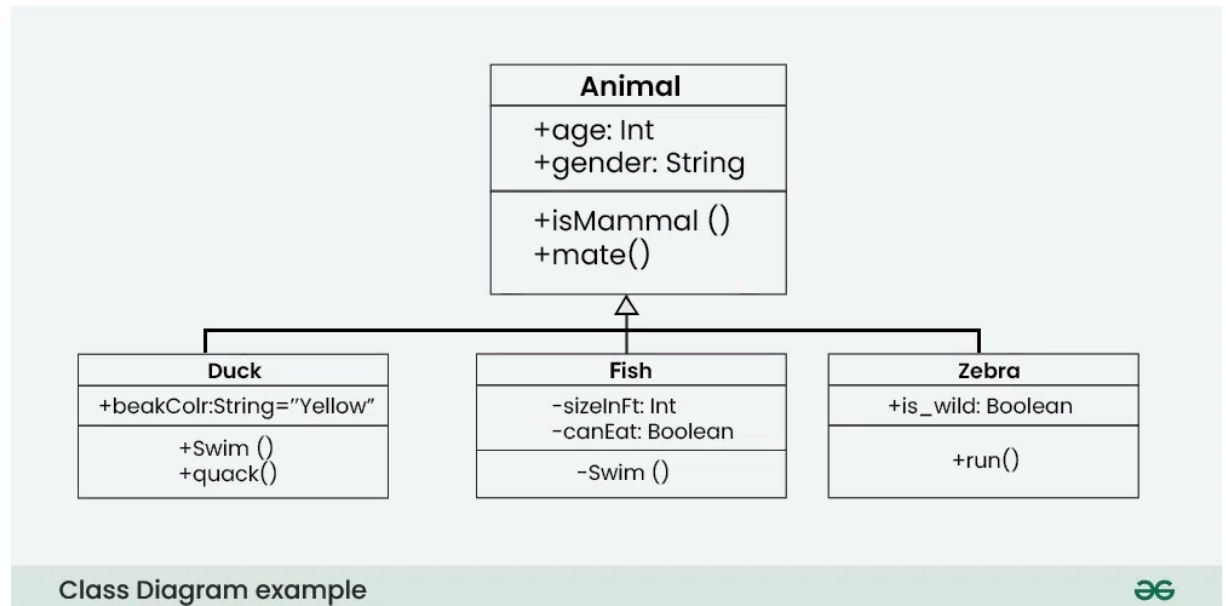
produzidas, a ausência de conflitos, o respeito à disponibilidade dos professores e a adequação da capacidade das salas devem ser verificados sistematicamente. Adicionalmente, testes de desempenho com volumes crescentes de dados são conduzidos para avaliar a escalabilidade do sistema e identificar seus limites operacionais, conforme descrito nos objetivos específicos deste trabalho.

3.4 Tecnologias Utilizadas

3.4.1 Banco de Dados Relacional

O modelo relacional, proposto por Codd (1970), organiza os dados em tabelas compostas por linhas e colunas, estabelecendo relações entre entidades por meio de chaves primárias e estrangeiras. A integridade dos dados é garantida por restrições declarativas no esquema, como unicidade, não nulidade e integridade referencial, que impedem a inserção de registros inconsistentes sem necessidade de validação exclusiva na camada de aplicação. Para documentar a estrutura do sistema, adota-se o Diagrama de Classes da UML (*Unified Modeling Language*), notação padronizada proposta por Booch, Rumbaugh e Jacobson (1999) que representa graficamente as classes do domínio, seus atributos, métodos e os relacionamentos entre elas, servindo como base para a implementação tanto da camada de persistência quanto da lógica de negócio do sistema. A Figura 5 ilustra um exemplo de Diagrama de Classes UML, demonstrando a notação utilizada para representar classes, atributos, métodos e relações de herança entre entidades.

Figura 5 — Exemplo de Diagrama de Classes UML



Fonte: GeeksForGeeks (2024)

3.4.2 PostgreSQL

O PostgreSQL é um sistema gerenciador de banco de dados objeto-relacional de código aberto, amplamente reconhecido por sua conformidade com o padrão SQL, sua robustez e confiabilidade no tratamento de dados, além do suporte a tipos de dados avançados e mecanismos de extensão (STONEBRAKER; ROWE, 1986). Um de seus principais diferenciais é o suporte às propriedades ACID, que garantem a segurança das transações: Atomicidade (uma operação é executada completamente ou não é executada), Consistência (os dados permanecem válidos após cada operação), Isolamento (transações simultâneas não interferem umas nas outras) e Durabilidade (os dados persistem mesmo após falhas do sistema).

No contexto do TimeForge, o PostgreSQL é utilizado para armazenar todas as entidades do domínio acadêmico e a grade horária gerada. Restrições de unicidade são definidas diretamente no banco de dados para evitar conflitos de alocação, como a sobreposição de professores, turmas ou salas em um mesmo horário, reforçando as garantias do algoritmo com uma camada adicional de consistência no nível do banco. A escolha do PostgreSQL se justifica por sua maturidade técnica, sua ampla adoção em sistemas acadêmicos e corporativos, sua integração nativa com o ecossistema Spring Boot via Spring

Data JPA e por ser uma solução de código aberto com suporte ativo e documentação consolidada.

3.4.3 Arquitetura RESTful

A transferência de estado representacional (*Representational State Transfer* — REST) é um estilo arquitetural para sistemas distribuídos proposto por Fielding (2000) em sua tese de doutorado. REST não é um protocolo nem um padrão, mas um conjunto de princípios arquiteturais que, quando aplicados ao desenvolvimento de APIs sobre o protocolo HTTP, resultam no que se denomina API RESTful.

Em uma API RESTful, cada recurso do sistema (como professor, turma ou sala) é identificado por uma URI (*Uniform Resource Identifier*) única e estável. As operações sobre esses recursos são expressas pelos métodos HTTP: GET para consulta, POST para criação, PUT para atualização completa e DELETE para remoção. As respostas são retornadas geralmente no formato JSON (JavaScript Object Notation), um formato leve e amplamente suportado para troca de dados entre sistemas (RICHARDSON; RUBY, 2007).

Entre os princípios fundamentais da arquitetura REST destacam-se a ausência de estado (*statelessness*), que determina que cada requisição deve conter todas as informações necessárias para ser processada pelo servidor, sem dependência de sessões armazenadas; a interface uniforme, que padroniza a comunicação entre cliente e servidor independentemente da implementação interna; e a separação clara entre cliente e servidor, que permite que ambos evoluam de forma independente. Essas características conferem às APIs RESTful alta escalabilidade, manutenibilidade e interoperabilidade (FIELDING, 2000).

No TimeForge, a API RESTful desenvolvida com Spring Boot expõe os *endpoints* responsáveis pelo cadastro e gerenciamento das entidades acadêmicas, bem como pelo acionamento do algoritmo de geração de horários, sendo consumida diretamente pela interface web.

3.4.4 Spring Boot

Um *framework* é uma estrutura de software reutilizável que fornece um conjunto de bibliotecas, convenções e abstrações sobre as quais uma aplicação pode ser construída, reduzindo a quantidade de código repetitivo que o desenvolvedor precisa escrever e estabelecendo uma arquitetura base para o projeto (PRESSMAN; MAXIM, 2016). Em vez de construir tudo do zero, o desenvolvedor utiliza o *framework* como fundação, concentrando esforços na lógica específica do sistema.

O Spring é um dos *frameworks* mais consolidados do ecossistema Java, oferecendo soluções para injeção de dependências, acesso a dados, segurança e desenvolvimento web (WALLS, 2019). Sobre ele, o Spring Boot introduz o princípio de convenção sobre configuração (*convention over configuration*), pelo qual o *framework* adota automaticamente configurações padrão sensatas para a maioria dos cenários, eliminando a necessidade de arquivos extensos de configuração manual e permitindo que uma aplicação funcional seja inicializada com pouquíssimo esforço de setup.

Para a persistência de dados, o Spring Boot integra-se nativamente ao Spring Data JPA, uma camada de abstração sobre o protocolo JPA (Java Persistence API) que permite mapear classes Java diretamente a tabelas do banco de dados relacional (técnica denominada ORM (*Object-Relational Mapping*)) e executar operações de consulta e escrita sem a necessidade de escrever SQL manualmente para os casos mais comuns (WALLS, 2019).

No TimeForge, o Spring Boot é responsável por expor os endpoints da API RESTful, orquestrar a execução do algoritmo de geração de horários e intermediar toda a comunicação com o banco de dados PostgreSQL por meio do Spring Data JPA, estruturando a aplicação em camadas bem definidas de controle, serviço e repositório. A escolha do Spring Boot se justifica pela sua produtividade no desenvolvimento de APIs RESTful em Java, pela robustez do ecossistema Spring para aplicações corporativas, pela integração nativa com PostgreSQL e Swagger, e pela ampla adoção da plataforma Java em sistemas de informação institucionais.

3.4.5 Vue.js

No desenvolvimento de aplicações web modernas, o *frontend* (camada responsável pela interface visual com a qual o usuário interage diretamente) é construído com o auxílio de *frameworks* JavaScript que organizam e simplificam a criação de interfaces dinâmicas e reativas. Vue.js é um *framework* JavaScript progressivo para a construção de interfaces de usuário, criado por Evan You em 2014 (YOU, 2014). O termo "progressivo" indica que o Vue.js pode ser adotado de forma gradual: desde a adição de interatividade a páginas simples até a construção de aplicações completas de página única (*Single Page Application* — SPA), nas quais toda a navegação ocorre sem recarregamento da página pelo navegador.

A arquitetura do Vue.js baseia-se no padrão Model-View-ViewModel (MVVM), que promove a separação entre o estado da aplicação (Model), a interface visual (View) e a lógica de apresentação que os conecta (ViewModel). Essa separação favorece a organização do código e facilita a manutenção do sistema à medida que ele cresce. O Vue.js estrutura as interfaces em componentes reutilizáveis e independentes, cada um encapsulando seu próprio

template HTML, estilo CSS e lógica JavaScript, o que permite que partes da interface sejam desenvolvidas, testadas e reutilizadas de forma isolada (YOU, 2014).

Uma das características centrais do Vue.js é a reatividade declarativa: quando o estado interno da aplicação é modificado, a interface é atualizada automaticamente de forma eficiente, sem necessidade de manipulação direta do DOM (*Document Object Model* — a representação estrutural da página no navegador). A comunicação com o servidor ocorre por meio de requisições HTTP assíncronas, que permitem que a interface consuma os dados retornados pela API RESTful sem bloquear a experiência do usuário.

No TimeForge, o Vue.js é responsável pela interface web destinada aos coordenadores acadêmicos, permitindo o cadastro e gerenciamento das entidades do sistema, o acionamento do processo de geração de horários e a visualização estruturada das grades produzidas pelo algoritmo. A escolha do Vue.js se justifica pela sua curva de aprendizado acessível em comparação a frameworks como React e Angular, pela sua arquitetura baseada em componentes que favorece a organização e a manutenibilidade da interface, e pela sua adequação natural ao consumo de APIs RESTful, tornando-o uma escolha eficiente para o desenvolvimento da camada de apresentação do sistema.

4 METODOLOGIA

Esta seção descreve as etapas, ferramentas e decisões técnicas utilizadas no desenvolvimento do TimeForge, seguindo o processo iterativo e incremental apresentado na fundamentação teórica. O desenvolvimento foi organizado em fases conectadas, nas quais os resultados de cada etapa serviram de base para a próxima, com ajustes contínuos ao longo de todo o processo.

4.1 Ferramentas e Ambiente de Desenvolvimento

O desenvolvimento do TimeForge adotou um conjunto de ferramentas consolidadas na indústria de software. O versionamento do código-fonte foi realizado com Git, utilizando o Forge URICER: Plataforma GIT institucional da URI Erechim como repositório remoto, o que permitiu o controle incremental das alterações, a rastreabilidade das decisões de implementação e a colaboração entre os membros da equipe.

A modelagem do banco de dados foi conduzida diretamente no PostgreSQL, aproveitando seus recursos nativos de definição de esquemas, restrições de integridade e visualização das estruturas relacionais. O diagrama de classes do sistema foi elaborado com o auxílio do draw.io, uma ferramenta de modelagem visual baseada em navegador que permite

criar diagramas UML de forma colaborativa e exportá-los em diferentes formatos, representando graficamente as entidades do domínio, seus atributos e relacionamentos antes da implementação.

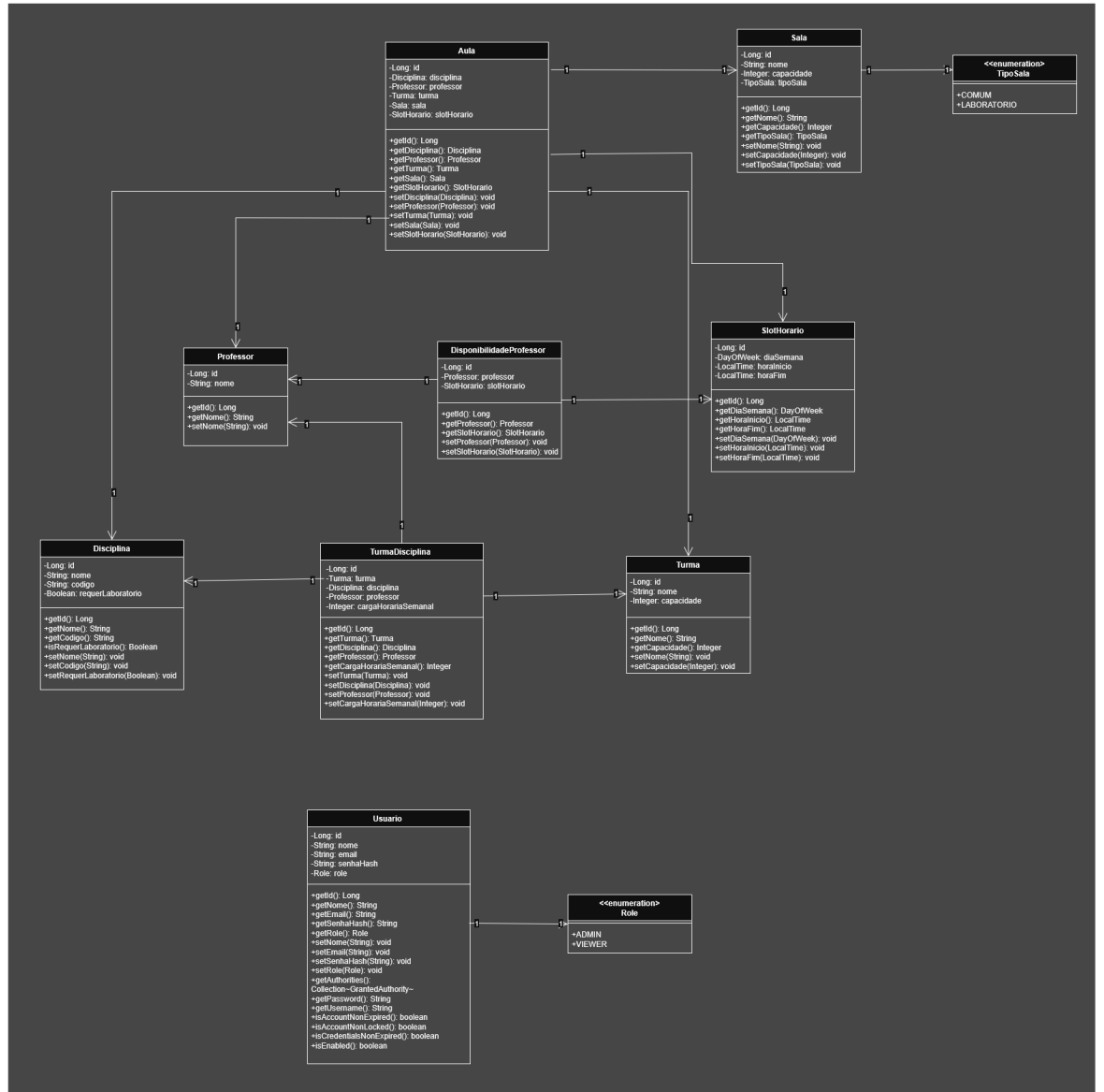
Para o desenvolvimento e validação da API RESTful, utilizou-se o Postman como ferramenta de teste de *endpoints*, possibilitando a verificação sistemática das requisições HTTP e dos retornos esperados. A documentação técnica da API foi gerada com Swagger/OpenAPI, integrado ao Spring Boot por meio da biblioteca SpringDoc, produzindo uma interface interativa que descreve todos os endpoints disponíveis, seus parâmetros e modelos de dados.

4.2 Modelagem do Banco de Dados e Diagrama de Classes

A primeira etapa do desenvolvimento consistiu na modelagem do domínio acadêmico. As entidades do sistema — Professor, Turma, Disciplina, Sala, SlotHorario, DisponibilidadeProfessor, TurmaDisciplina e Aula — foram identificadas a partir do levantamento de requisitos e estruturadas em um esquema relacional no PostgreSQL. Restrições de unicidade e integridade referencial foram definidas diretamente no banco de dados, prevenindo conflitos de alocação em nível de persistência.

Em paralelo, o diagrama de classes UML foi elaborado para representar as entidades do sistema sob a perspectiva orientada a objetos, documentando os atributos, métodos e relacionamentos entre as classes que compõem tanto a camada de domínio quanto a camada de persistência da aplicação. Conforme ilustrado na Figura 6, cada entidade do domínio acadêmico é representada como uma classe com seus respectivos atributos e relacionamentos, evidenciando as dependências estruturais do sistema. Essa modelagem serviu como contrato entre a estrutura do banco de dados e a implementação em Java com Spring Boot.

Figura 6 — Diagrama de Classes do Sistema TimeForge

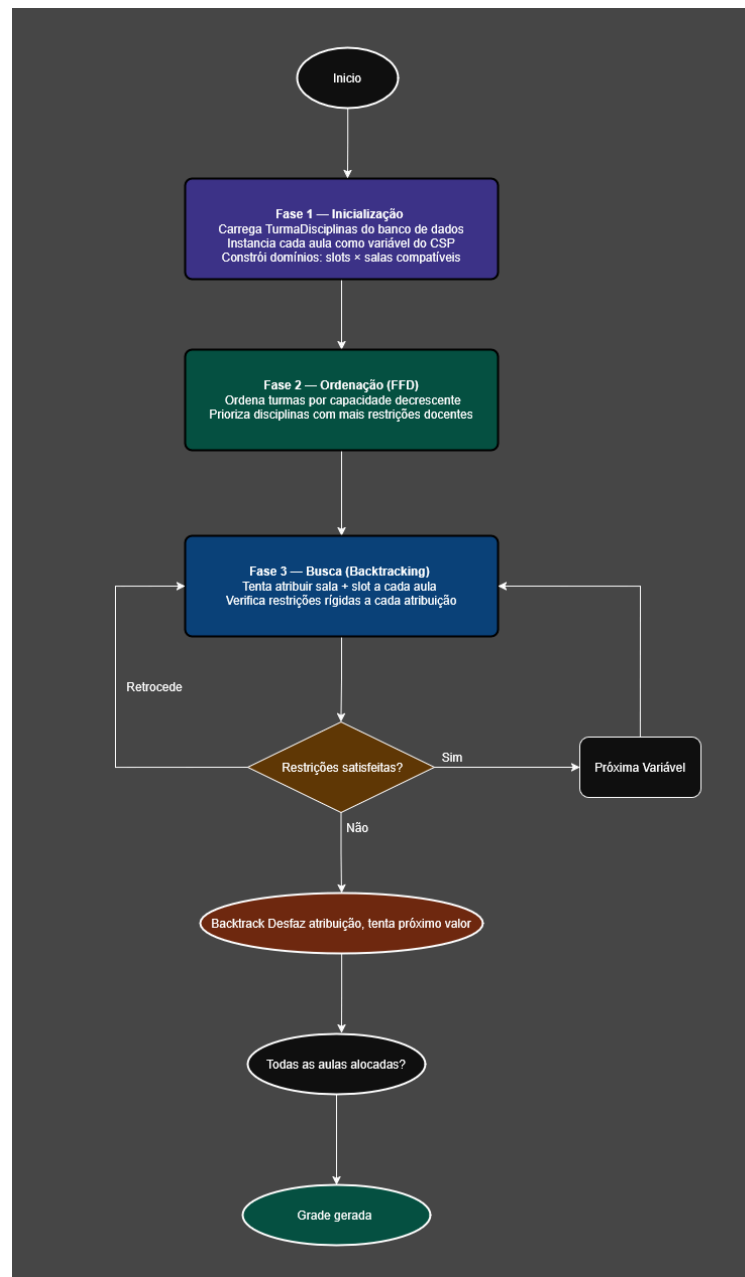


Fonte: Os autores (2026)

4.3 Implementação do Algoritmo de Geração de Horários

O núcleo algorítmico do TimeForge foi implementado em Java, estruturado como um serviço dedicado dentro da arquitetura Spring Boot. O algoritmo opera em três fases sequenciais, conforme ilustrado no fluxograma da Figura 7.

Figura 7 — Fluxograma do Algoritmo de Geração de Horários do TimeForge



Fonte: Os autores (2026)

Na fase de inicialização, o sistema carrega do banco de dados todas as TurmaDisciplinas cadastradas, cada uma representando uma oferta curricular com sua respectiva carga horária semanal. Cada aula a ser alocada é instanciada como uma variável do CSP, e os domínios são construídos pelo produto cartesiano entre os slots de horário disponíveis e as salas com capacidade compatível com a turma.

Na fase de ordenação das variáveis limitadoras, aplica-se a heurística MRV (Minimum Remaining Values): aulas que exigem laboratório ou cujos professores possuem maior

restrição de horários são priorizadas, reduzindo a probabilidade de impasse. Em paralelo, os domínios de salas seguem a heurística FFD (First Fit Decreasing), sendo ordenados da maior para a menor capacidade.

Na fase de busca, o algoritmo de Backtracking percorre as variáveis ordenadas, tentando atribuir a cada aula um par válido de sala e slot de horário. A cada atribuição, as restrições rígidas do sistema são verificadas: ausência de conflito de professor, turma e sala no mesmo slot, e conformidade com a disponibilidade declarada do docente. Quando uma inconsistência é detectada, o algoritmo desfaz a atribuição e tenta o próximo valor do domínio, retrocedendo na árvore de decisão conforme necessário.

A implementação completa do algoritmo encontra-se disponível no arquivo `ScheduleGenerator.java`, acessível em https://git.uricer.edu.br/111624/TimeForge_Backend/src/branch/main/timeforge-api/src/main/java/br/com/timeforge/timeforge_api/engine/ScheduleGenerator.java.

4.4 Desenvolvimento da API RESTful

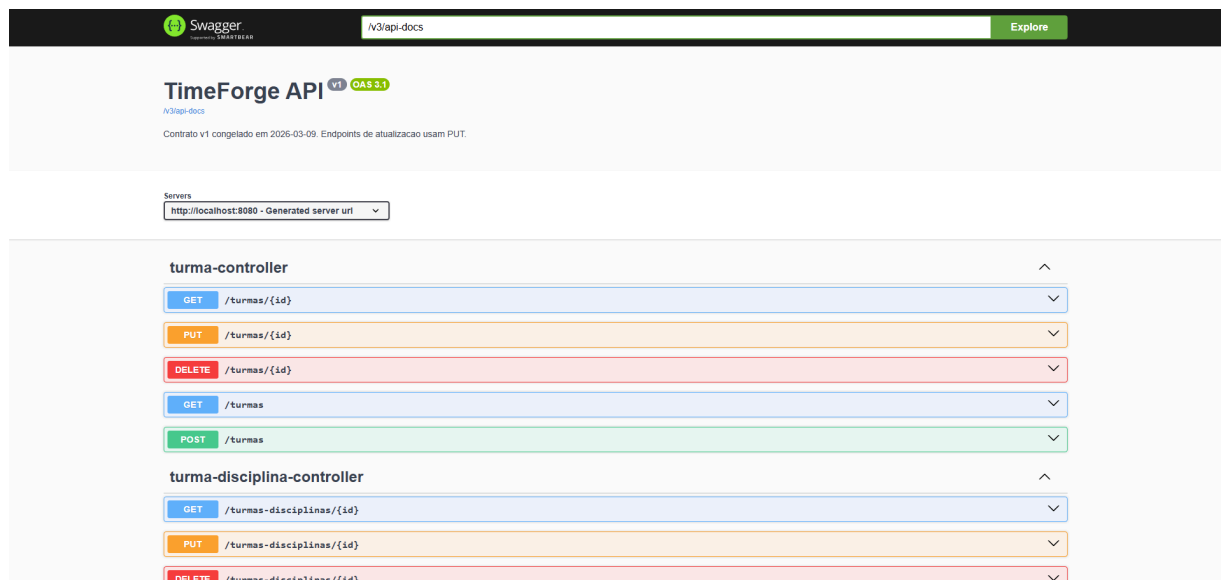
A API RESTful foi desenvolvida com Spring Boot, estruturada em quatro camadas bem definidas: a camada de controle (Controller), responsável por receber e responder às requisições HTTP; a camada de serviço (Service), onde reside a lógica de negócio e a orquestração do algoritmo; a camada de repositório (Repository), responsável pela comunicação com o banco de dados PostgreSQL via Spring Data JPA; e a camada de transferência de dados (Data Transfer Object - DTO), que define objetos específicos para o tráfego de informações entre o cliente e a API, desacoplando a representação externa dos dados da estrutura interna das entidades do domínio e evitando a exposição desnecessária de informações sensíveis do modelo.

Para cada entidade do domínio foram implementados endpoints que seguem as convenções REST, permitindo operações de criação, consulta, atualização e remoção. Um endpoint específico foi dedicado ao acionamento do processo de geração de horários, recebendo os parâmetros de configuração e retornando a grade produzida pelo algoritmo.

Toda a API foi documentada com Swagger/OpenAPI, integrado ao Spring Boot por meio da biblioteca SpringDoc, gerando uma interface interativa acessível durante o desenvolvimento e utilizada como documentação técnica oficial do sistema. Conforme ilustrado na Figura 8, a interface do Swagger apresenta de forma organizada todos os endpoints disponíveis, seus métodos HTTP, parâmetros de entrada, modelos de dados e

exemplos de resposta, permitindo que desenvolvedores testem e compreendam a API diretamente pelo navegador, sem necessidade de ferramentas externas.

Figura 8 — Interface do Swagger com os Endpoints da API TimeForge



Fonte: Os autores (2026)

4.5 Desenvolvimento da Interface Web

A interface web do TimeForge foi desenvolvida em Vue.js, estruturada em componentes reutilizáveis organizados por funcionalidade. A comunicação com a API RESTful ocorre por meio de requisições HTTP assíncronas, utilizando a biblioteca Axios para o gerenciamento das chamadas e o tratamento de respostas e erros.

A interface oferece aos coordenadores acadêmicos telas para o cadastro e gerenciamento de todas as entidades do sistema — professores, turmas, disciplinas, salas e slots de horário —, além de uma tela dedicada à visualização da grade horária gerada, apresentando as aulas alocadas de forma estruturada e de fácil leitura.

A interface web, ao centralizar as operações de gerenciamento e visualização em um único ambiente, complementa o backend desenvolvido e viabiliza o uso do TimeForge por coordenadores acadêmicos sem necessidade de conhecimento técnico sobre a API subjacente.

4.6 Avaliação de Desempenho e Escalabilidade

Com o objetivo de identificar os limites operacionais do sistema, foram conduzidos testes de desempenho com volumes crescentes de dados. A estratégia adotada consistiu em

definir cinco cenários sintéticos de complexidade progressiva, variando simultaneamente o número de turmas, professores, salas e slots de horário disponíveis, de forma a simular instituições de diferentes portes e pressionar o algoritmo até o ponto em que seu desempenho se torna inviável para uso em produção.

Os cinco cenários definidos para os testes estão descritos na Quadro 1:

Quadro 1 — Cenários Sintéticos para Avaliação de Escalabilidade do TimeForge

Cenário	Porte	Turmas	Professores	Salas	Slots Semanais	Aulas Esperadas (aprox.)
C1	Muito Pequeno	5	10	5	25	50
C2	Pequeno	10	20	10	25	100
C3	Médio	25	40	20	30	250
C4	Grande	50	70	35	30	500
C5	Muito Grande	100	120	60	35	1000

Fonte: Os autores (2026)

Para cada cenário, os dados foram inseridos de forma sintética diretamente no banco de dados, garantindo que as disponibilidades dos professores e as capacidades das salas fossem proporcionais ao volume de turmas, evitando inviabilidade trivial por escassez de recursos. O algoritmo foi executado três vezes para cada cenário, registrando a média dos resultados a fim de reduzir variações decorrentes do ambiente de execução.

As métricas registradas em cada execução foram o tempo total de processamento do algoritmo em milissegundos, a taxa de sucesso expressa como percentual de aulas efetivamente alocadas em relação ao total esperado, e o ponto de travamento, definido como o cenário a partir do qual o tempo de execução ultrapassa um limiar considerado inaceitável para uso interativo ou no qual a taxa de sucesso cai abruptamente, indicando saturação combinatória do espaço de busca.

Os resultados obtidos são apresentados e discutidos na seção de Resultados, onde as métricas coletadas para cada cenário são comparadas e analisadas em relação ao comportamento esperado do algoritmo de Backtracking sob crescente pressão combinatória.

5 ANÁLISE DOS RESULTADOS

Esta seção apresenta os resultados obtidos a partir da implementação do TimeForge, organizados em dois eixos: as funcionalidades entregues pelo sistema como produto e o desempenho e a escalabilidade do algoritmo de geração de horários, avaliados por meio dos cenários sintéticos definidos na metodologia.

O código-fonte completo do sistema, composto pelo repositório do backend disponível em https://git.uricer.edu.br/111624/TimeForge_Backend e pelo repositório do frontend disponível em https://git.uricer.edu.br/111624/TimeForge_Frontend, está hospedado na plataforma institucional Forge URICER. O projeto, incluindo ambos os módulos como submódulos Git, encontra-se em <https://www.forge.uricer.edu.br/2026-1-ProjetoIntegrador3/GrupoG>. O commit correspondente à versão final entregue está registrado em <https://www.forge.uricer.edu.br/2026-1-ProjetoIntegrador3/GrupoG/src/commit/1d69f33e57c6ea6c61212540cc47568a07bcf4f0>.

5.1 Funcionalidades Entregues

O TimeForge foi desenvolvido como um sistema completo, composto por backend, frontend e banco de dados relacional, cobrindo integralmente os objetivos específicos definidos na Seção 2.2. Esta seção descreve as decisões de implementação mais relevantes de cada camada do sistema.

5.1.1 Núcleo Algorítmico

O algoritmo de geração de horários é implementado na classe *ScheduleGenerator*, estruturada como um componente Spring gerenciado por injeção de dependências. Sua execução é organizada em seis etapas sequenciais, cada uma com responsabilidade bem delimitada.

Na primeira etapa, o sistema carrega as ofertas da turma a partir da entidade *TurmaDisciplina*, que associa uma turma a uma disciplina, um professor responsável e uma carga horária semanal. Cada oferta é validada estruturalmente antes de ingressar no fluxo do algoritmo: a ausência de professor, disciplina, turma ou carga horária inválida interrompe a

execução com uma exceção tipificada, evitando que dados inconsistentes contaminem a busca.

Na segunda etapa, cada oferta é expandida em variáveis individuais do CSP por meio do método *expandirVariaveisCsp*. Uma oferta com carga horária semanal de três aulas gera três variáveis independentes, cada uma representando uma aula a ser alocada. Esse mapeamento explicita o espaço de busca real do problema: o número total de variáveis é a soma das cargas horárias semanais de todas as ofertas da turma.

Na terceira etapa, os domínios de slots e salas são construídos. Os slots são ordenados cronologicamente por dia da semana e horário de início, garantindo que a busca percorra o espaço de forma determinística e auditável. As salas são ordenadas em ordem decrescente de capacidade, implementando a heurística FFD: salas maiores são oferecidas primeiro para cada tentativa de alocação, reduzindo a probabilidade de que turmas com maior número de alunos encontrem apenas salas insuficientes ao final do processo.

Na quarta etapa, as disponibilidades de cada professor são carregadas do banco de dados e indexadas em um mapa *professorId e Set<slotId>*. Essa estrutura permite verificação em tempo constante durante a busca, sem consultas adicionais ao banco por iteração.

Na quinta etapa, as variáveis são ordenadas pela heurística MRV: professores com menor número de slots disponíveis têm suas aulas priorizadas na ordem de atribuição. Variáveis que exigem laboratório são também antecipadas, pois possuem domínio de salas mais restritas. Essa ordenação reduz o branching factor nas primeiras decisões da árvore de busca, onde o custo de um retrocesso tardio seria maior.

Na sexta etapa, o algoritmo de Backtracking percorre as variáveis ordenadas. Para cada variável, itera sobre os slots do domínio e, para cada slot compatível, itera sobre as salas possíveis em ordem FFD. A cada tentativa, três restrições rígidas são verificadas: ausência de sobreposição temporal do professor, ausência de sobreposição temporal da turma e ausência de sobreposição temporal da sala. A verificação de sobreposição é baseada em intervalos reais de *horaInicio* e *horaFim*, não apenas em identidade de slot, o que garante que slots de horários distintos mas com intervalos coincidentes também sejam corretamente detectados como conflitantes. Quando uma tentativa é bem-sucedida, a ocupação dos três recursos é registrada nas estruturas *ocupacaoProfessor*, *ocupacaoTurma* e *ocupacaoSala*, e o algoritmo avança recursivamente para a próxima variável. Quando uma inconsistência é detectada em qualquer nível da recursão, o algoritmo desfaz as três ocupações registradas (rollback) e tenta o próximo valor do domínio.

Dois mecanismos de segurança limitam a execução em cenários de alta pressão combinatória: um limite de 200.000 iterações e um timeout de dez segundos, ambos verificados a cada chamada recursiva pelo objeto *BuscaEstado*. Quando qualquer um dos limites é excedido, o algoritmo interrompe a busca e retorna a melhor solução parcial encontrada até aquele ponto, registrada incrementalmente ao longo da execução. Esse comportamento garante que o sistema nunca bloqueie indefinidamente e sempre entregue um resultado, ainda que parcial, ao coordenador acadêmico.

O algoritmo considera ainda as aulas já persistidas de outras turmas como restrições fixas: antes de iniciar a busca, as ocupações existentes no banco de dados são carregadas e aplicadas às estruturas de ocupação, impedindo que a nova grade conflite com grades já geradas para outras turmas da mesma escola.

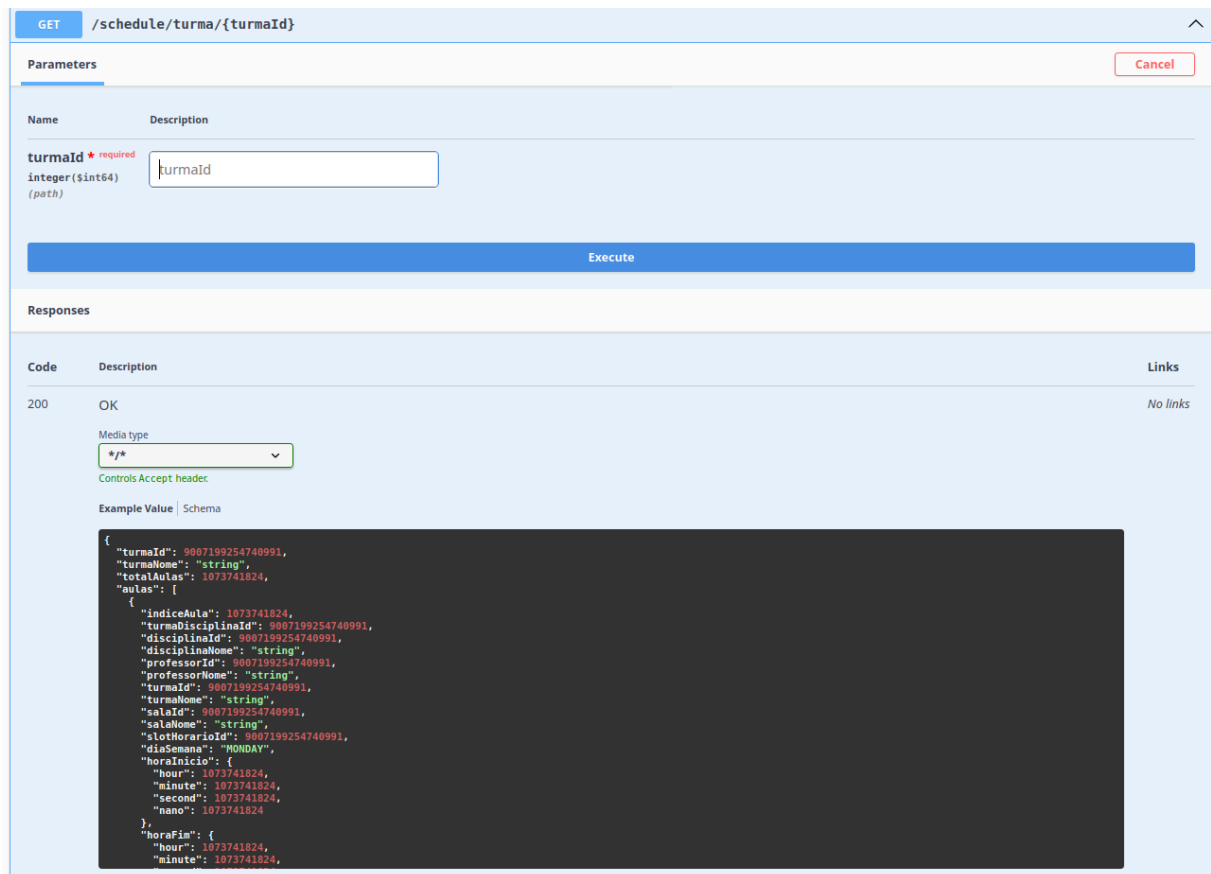
5.1.2 API RESTful e Camada de Serviço

API RESTful é exposta pelo *ScheduleController*, mapeado em */schedule*, que oferece cinco endpoints principais. A geração de horários é acessível tanto via POST */schedule/generate* com corpo JSON contendo o *turmaId*, quanto via POST */schedule/generate/{turmaId}* com o identificador diretamente no caminho, ambos sujeitos a validação com Bean Validation antes de atingir a camada de serviço. A consulta da grade persistida é realizada via GET */schedule/turma/{turmaId}*, e as exportações são disponibilizadas em PDF via GET */schedule/turma/{turmaId}/{formato}/export/pdf* e em Excel via GET */schedule/turma/{turmaId}/export/excel*. Os arquivos exportados são retornados como *ResponseEntity<byte[]>* com os cabeçalhos *Content-Disposition* e *Content-Type* adequados, permitindo download direto pelo navegador sem redirecionamentos adicionais.

A resposta do endpoint de geração é estruturada pelo DTO *ScheduleGenerationResponseDTO*, que inclui o indicador de sucesso, a mensagem de resultado, o total de aulas necessárias, o total de aulas efetivamente alocadas, o tempo de execução do Backtracking em milissegundos e a lista detalhada de aulas alocadas, cada uma contendo disciplina, professor, turma, sala, dia da semana, horário de início e horário de fim. O campo *observacoes* acumula alertas gerados durante a execução, como professores sem disponibilidade cadastrada, salas sem capacidade compatível ou interrupção por limite de iterações, fornecendo rastreabilidade diagnóstica do processo ao coordenador.

A Figura 9 apresenta a interface do Swagger com os endpoints do ScheduleController e dos demais recursos da API, demonstrando a documentação interativa gerada automaticamente pela biblioteca SpringDoc.

Figura 9 —Interface do Swagger com os endpoints da API RESTful do TimeForge



Fonte: Os autores (2026)

5.1.3 Controle de Acesso e Isolamento por Escola

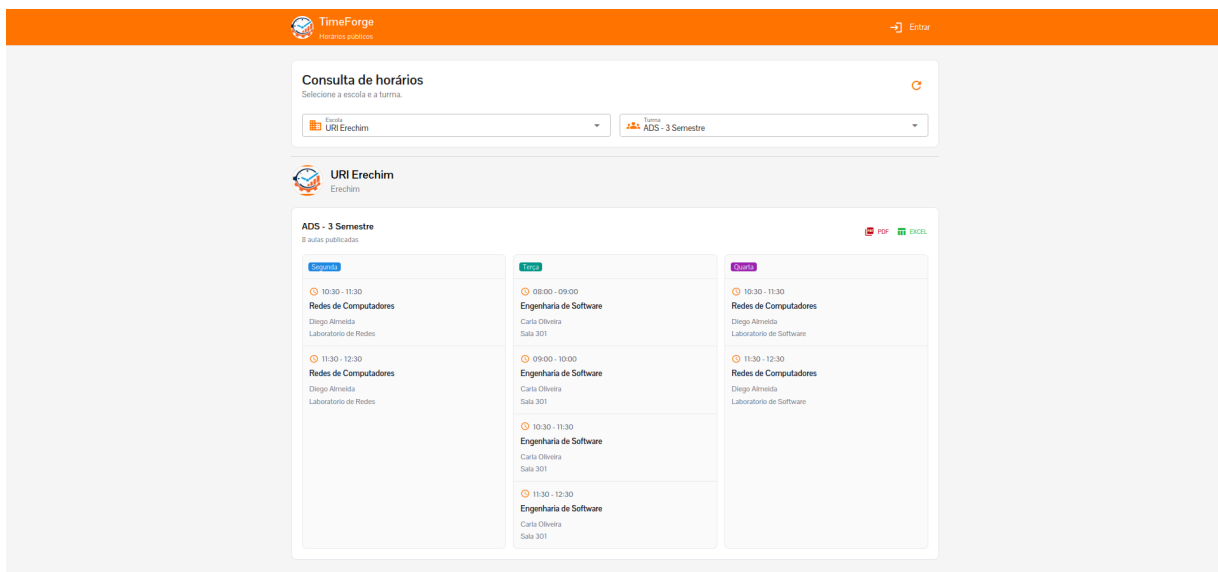
O sistema implementa controle de acesso baseado em perfis via Spring Security com autenticação JWT. O perfil *SUPER_ADMIN* possui acesso irrestrito a todas as escolas cadastradas, podendo operar sobre qualquer instituição mediante informação do parâmetro *escolaId*. O perfil *ADMIN_ESCOLA* tem seu escopo restrito automaticamente pelo backend à escola vinculada ao usuário autenticado, sem necessidade de qualquer configuração no frontend. O isolamento entre escolas é reforçado em nível de repositório: todas as consultas de dados acadêmicos são filtradas por *escolaId*, impedindo que dados de uma instituição sejam acessados ou modificados por usuários de outra, mesmo em caso de manipulação direta da requisição.

5.1.4 Interface Web

O frontend, desenvolvido com Vue 3, Quasar Framework e Pinia para gerenciamento de estado, organiza-se em duas áreas funcionais distintas. A área administrativa, acessível mediante autenticação JWT, oferece telas de CRUD para todas as entidades do domínio acadêmico: professores, disciplinas, turmas, salas, slots de horário, disponibilidades docentes e vínculos turma-disciplina-professor. A geração e a consulta de grades são realizadas por telas dedicadas, com acionamento direto do endpoint de geração e visualização estruturada do resultado. A área pública, acessível sem autenticação no caminho `/horarios/{slug}`, permite que alunos e professores consultem a grade de qualquer turma pelo endereço da escola, sem necessidade de cadastro ou login.

A Figura 10 apresenta a tela de visualização da grade horária gerada para uma turma, exibindo as aulas alocadas organizadas por dia da semana e horário.

Figura 10 —Tela de visualização da grade horária gerada pelo TimeForge



Fonte: Os autores (2026)

A comunicação entre o frontend e a API é centralizada na camada de serviços (`src/services`), que encapsula todas as chamadas Axios e isola as telas da lógica de autenticação e tratamento de erros. O token JWT é armazenado na sessão local do navegador e anexado automaticamente às requisições protegidas pelo interceptor configurado no `api-client.js`.

5.1.5 Banco de Dados e Persistência

O banco de dados PostgreSQL armazena todas as entidades do domínio acadêmico com restrições de unicidade e integridade referencial definidas diretamente no esquema, reforçando em nível de persistência as garantias produzidas pelo algoritmo. A geração de horários opera em memória e retorna o resultado ao cliente sem persistência automática, cabendo ao coordenador acadêmico confirmar e salvar a grade produzida. Essa decisão de projeto preserva a rastreabilidade das versões de grade e evita sobrescrita acidental de alocações previamente confirmadas. O acesso ao banco é intermediado pelo Spring Data JPA, com repositórios tipados para cada entidade do domínio, eliminando SQL manual para as operações de leitura e escrita mais comuns e concentrando as consultas customizadas em métodos de interface com nomenclatura derivada.

5.2 Desempenho e Escalabilidade do Algoritmo

Os testes de desempenho foram executados sobre a implementação Java do núcleo algorítmico, utilizando o framework JUnit integrado ao Maven. Cada cenário foi executado três vezes, registrando-se a média dos tempos de processamento e das taxas de alocação, conforme descrito na Seção 4.6. Os resultados obtidos para os cinco cenários principais estão consolidados no Quadro 2.

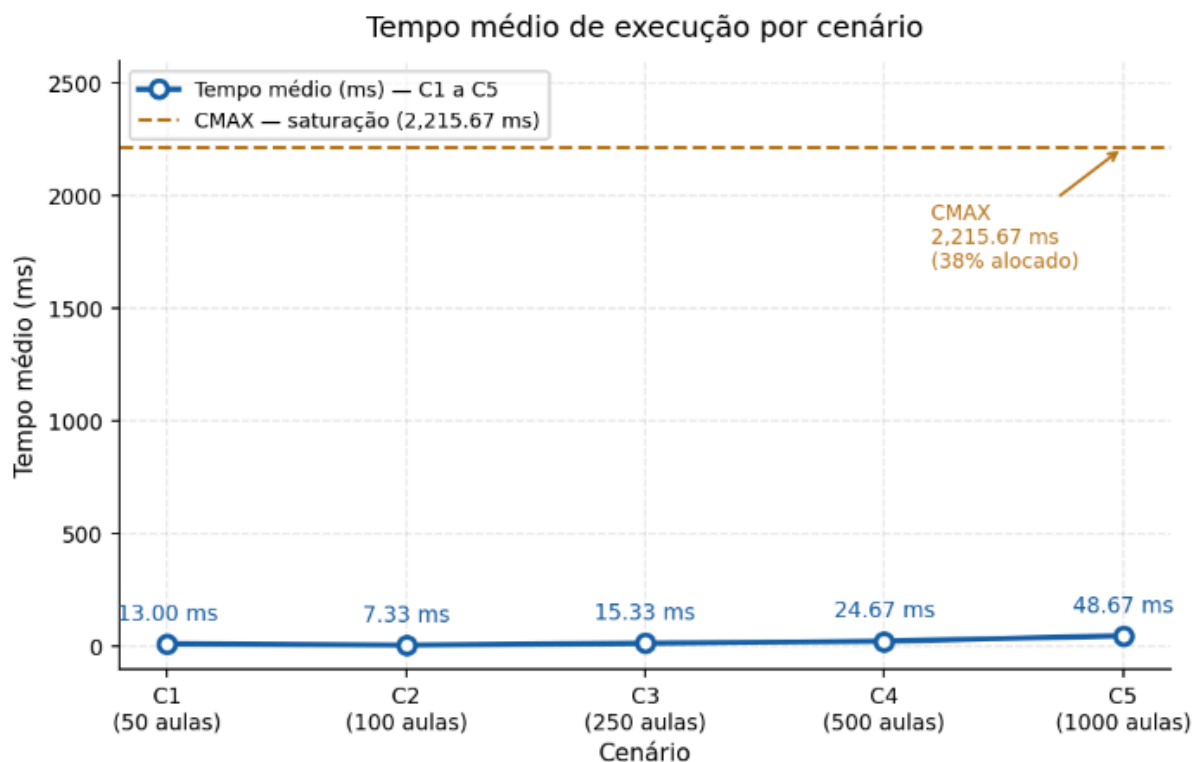
Quadro 2 — Resultados dos Testes de Desempenho por Cenário

Cenário	Porte	Turmas	Professores	Salas	Slots Semanais	Aulas Esp.	Tempo Méd. (ms)	Sucesso (%)
C1	Muito Pequeno	5	10	5	25	50	13,00	100,00
C2	Pequeno	10	20	10	25	100	7,33	100,00
C3	Médio	25	40	20	30	250	15,33	100,00
C4	Grande	50	70	35	30	500	24,67	100,00
C5	Muito Grande	100	120	60	35	1000	48,67	100,00

Fonte: Os autores (2026)

A Figura 11 apresenta o comportamento do tempo médio de execução ao longo dos cenários C1 a C5, evidenciando o crescimento sublinear observado. A linha tracejada indica o tempo registrado no cenário de saturação CMAX (2.215,67 ms), posicionando-o como referência de degradação máxima em relação aos cenários regulares.

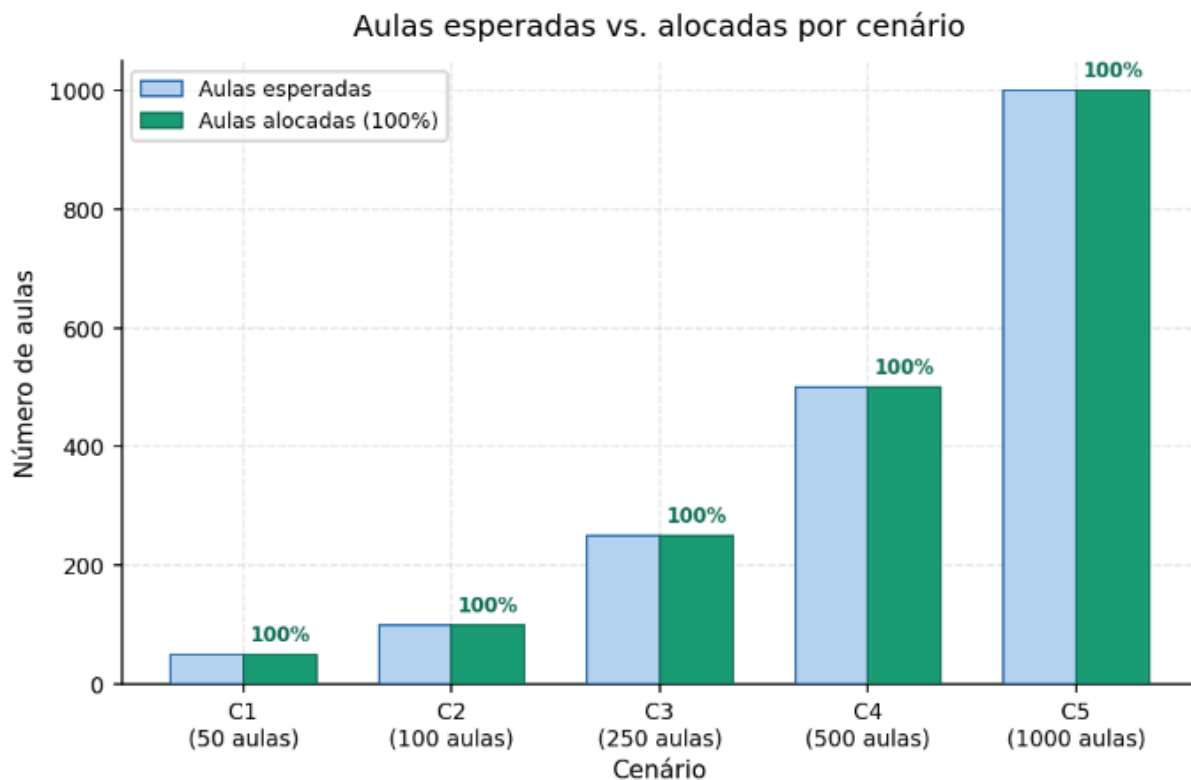
Figura 11 — Tempo médio de execução do algoritmo TimeForge por cenário (C1–C5) e referência do cenário de saturação CMAX



Fonte: Os autores (2026)

A Figura 12 apresenta a comparação entre o número de aulas esperadas e o número de aulas efetivamente alocadas em cada cenário, confirmando visualmente a correspondência integral entre as duas grandezas nos cenários C1 a C5.

Figura 12 — Aulas esperadas versus aulas alocadas por cenário (C1–C5)



Fonte: Os autores (2026)

Os resultados demonstram que o algoritmo híbrido atingiu uma taxa de alocação de 100% em todos os cinco cenários, alocando com sucesso desde 50 até 1.000 aulas sem nenhuma falha de atribuição. O tempo de processamento cresceu de forma sublinear em relação ao volume de aulas: enquanto o número de aulas aumentou vinte vezes entre C1 e C5 (de 50 para 1.000), o tempo médio cresceu aproximadamente 3,7 vezes (de 13,00 ms para 48,67 ms). Esse comportamento indica que as heurísticas MRV e FFD foram eficazes em reduzir o espaço de busca explorado pelo Backtracking, evitando a explosão combinatória típica de abordagens de força bruta para instâncias dessa magnitude.

Destaca-se o resultado do cenário C2, cujo tempo médio de 7,33 ms foi inferior ao do C1 (13,00 ms), ainda que possua o dobro de aulas. Esse comportamento é atribuído à maior densidade de recursos disponíveis no C2 (10 salas e 20 professores para 100 aulas) em relação ao C1 (5 salas e 10 professores para 50 aulas), o que resulta em domínios mais amplos para cada variável do CSP e, consequentemente, em menos operações de retrocesso durante a busca.

5.2.1 Cenário de Saturação (CMAX)

Com o objetivo de identificar o ponto de colapso do algoritmo, foi executado um cenário adicional de saturação intencional, denominado CMAX. Nesse cenário, o número de aulas esperadas (600) foi deliberadamente desproporcional aos recursos disponíveis: apenas 12 salas e 19 slots semanais para 30 turmas, configurando um problema estruturalmente inviável em sua completude. Os resultados do CMAX, obtidos sobre 60 repetições, estão apresentados no Quadro 3.

Quadro 3 — Resultado do Cenário de Saturação CMAX

Cenário	Porte	Turmas	Professores	Salas	Slots Semanais	Aulas Esp.	Tempo Méd. (ms)	Sucesso (%)
CMAX	Saturação	30	40	12	19	50	2.215,67	38,00

Fonte: Os autores (2026)

No cenário CMAX, o algoritmo alocou 38% das aulas esperadas em tempo médio de 2.215,67 ms (aproximadamente 2,2 segundos), entregando uma grade parcial como resultado de melhor esforço. Esse comportamento evidencia duas características centrais do sistema: primeiro, a degradação de desempenho do Backtracking sob alta pressão combinatória, confirmando o comportamento teórico esperado para problemas NP-difíceis em instâncias com recursos insuficientes; segundo, a capacidade do TimeForge de produzir grades parciais em vez de simplesmente falhar, aliviando o trabalho manual dos coordenadores mesmo em cenários de inviabilidade estrutural.

O tempo de 2,2 segundos ainda se mantém dentro de um limiar aceitável para uso interativo, considerando que o cenário CMAX representa uma situação deliberadamente inviável, com relação de aproximadamente 31,6 aulas por slot disponível, muito além do que qualquer instituição real operária.

6 CONSIDERAÇÕES FINAIS

Este trabalho apresentou o desenvolvimento do TimeForge, um sistema computacional para geração automática de grades horárias acadêmicas fundamentado em uma abordagem híbrida que combina modelagem via Constraint Satisfaction Problem (CSP), busca por Backtracking com retrocesso inteligente e a heurística First Fit Decreasing (FFD) para alocação de recursos físicos. O sistema foi implementado sobre uma arquitetura composta por backend em Java 21 com Spring Boot, interface web desenvolvida com Vue 3 e Quasar

Framework, e banco de dados relacional PostgreSQL, integrando as camadas de persistência, lógica de negócio e apresentação em um produto funcional e coeso.

Todos os seis objetivos específicos definidos na Seção 2.2 foram integralmente atingidos. O domínio acadêmico foi modelado em um esquema relacional persistido no PostgreSQL, com restrições de unicidade e integridade referencial definidas diretamente no banco, refletindo fielmente as entidades e restrições do ambiente institucional real. O núcleo algorítmico foi implementado combinando CSP, Backtracking com rollback e heurísticas MRV e FFD, operando sobre as entidades cadastradas e respeitando as restrições rígidas do problema. A API RESTful foi desenvolvida com Spring Boot em arquitetura de quatro camadas: Controller, Service, Repository e DTO, documentada via Swagger/OpenAPI e validada com Bean Validation. A interface web foi construída em Vue 3 com Quasar Framework, oferecendo área administrativa completa para coordenadores e área pública de consulta de horários sem necessidade de autenticação. A documentação técnica da API foi gerada automaticamente pela biblioteca SpringDoc e disponibilizada em interface interativa acessível diretamente pelo navegador. Por fim, a avaliação de desempenho e escalabilidade foi conduzida por meio de cinco cenários sintéticos de complexidade progressiva, complementados por um cenário de saturação intencional.

Os resultados dos testes de desempenho demonstraram que o algoritmo híbrido atingiu taxa de alocação de 100% em todos os cenários regulares, processando desde 50 até 1.000 aulas em tempo máximo de 48,67 ms. O crescimento do tempo de execução em relação ao volume de aulas foi sublinear, evidenciando a eficácia das heurísticas MRV e FFD em reduzir o espaço de busca explorado pelo Backtracking. O cenário de saturação CMAX, projetado com recursos deliberadamente insuficientes para o volume de aulas demandado, revelou o comportamento do algoritmo sob pressão combinatória extrema: 38% de alocação em 2.215,67 ms, com entrega de grade parcial como resultado de melhor esforço em lugar de falha abrupta. Esse comportamento confirma a adequação da abordagem para uso em instituições de médio porte, onde a rastreabilidade das decisões e a interpretabilidade do processo são requisitos operacionais fundamentais.

Do ponto de vista prático, o TimeForge representa uma contribuição concreta ao cenário brasileiro de automação acadêmica, onde soluções integradas, de código aberto e voltadas a instituições de médio porte permanecem escassas. A combinação de robustez algorítmica, persistência relacional estruturada, controle de acesso por perfil e interface de uso intuitivo posiciona o sistema como uma alternativa viável à elaboração manual de horários, processo reconhecidamente suscetível a erros e de elevado custo operacional.

Como limitação principal, o algoritmo de Backtracking está sujeito à degradação de desempenho em instâncias com alta relação entre aulas demandadas e recursos disponíveis, comportamento inerente à classe NP-difícil do problema e mitigado, mas não eliminado, pelas heurísticas adotadas. Instâncias com restrições de disponibilidade muito rígidas ou com grande número de disciplinas que exigem laboratório podem pressionar o espaço de busca de forma desproporcional, exigindo ajuste manual dos parâmetros de timeout e limite de iterações configuráveis pelo administrador do sistema.

Como desdobramentos futuros, pretende-se validar o sistema em ambiente real mediante implantação em uma instituição de ensino parceira, coletando feedback de coordenadores acadêmicos para orientar refinamentos funcionais e de usabilidade. Em paralelo, planeja-se a publicação do sistema em infraestrutura de nuvem, tornando-o acessível a instituições sem estrutura de servidores própria. No plano funcional, identificam-se como extensões prioritárias o suporte a múltiplos semestres com histórico e comparação de grades entre períodos letivos, o envio de notificações automáticas por e-mail aos professores e coordenadores quando uma nova grade é gerada ou alterada, e o suporte a instituições com múltiplos campi, permitindo que um único sistema gerencie escolas com unidades geograficamente distribuídas e restrições de alocação entre campus. A incorporação de restrições flexíveis ao modelo CSP, como preferências de horário declaradas pelos docentes, representa uma extensão algorítmica natural que permitiria aproximar o sistema das competições ITC e ampliar sua aplicabilidade em contextos institucionais mais complexos.

7 REFERÊNCIAS

BONUTTI, A.; DE CESCO, F.; DI GASPERO, L.; SCHAERF, A. Benchmarking curriculum-based course timetabling: formulations, data formats, instances, validation, visualization, and results. *Annals of Operations Research*, v. 194, n. 1, p. 59–70, 2012. DOI: 10.1007/s10479-010-0707-0.

BOOCH, G. *Object-Oriented Analysis and Design with Applications*. 2. ed. Redwood City: Benjamin/Cummings, 1994.

BOOCH, G.; RUMBAUGH, J.; JACOBSON, I. *The Unified Modeling Language User Guide*. Reading: Addison-Wesley, 1999.

CODD, E. F. A relational model of data for large shared data banks. *Communications of the ACM*, v. 13, n. 6, p. 377–387, 1970.

COLORNI, A.; DORIGO, M.; MANIEZZO, V. Genetic algorithms and highly constrained problems: the time-table case. In: PARALLEL PROBLEM SOLVING FROM NATURE, 1., 1990, Dortmund. Anais[...]. Berlin: Springer, 1991. p. 55–59.

DECHTER, R. Constraint Processing. San Francisco: Morgan Kaufmann, 2003.

DI GASPERO, L. et al. The second international timetabling competition (ITC-2007): curriculum-based course timetabling track. In: PRACTICE AND THEORY OF AUTOMATED TIMETABLING, 7., 2008, Montreal. Anais[...]. 2007.

FIELDING, R. T. Architectural Styles and the Design of Network-based Software Architectures. 2000. Tese (Doutorado em Informação e Ciência da Computação) — University of California, Irvine, 2000. Disponível em: <https://ics.uci.edu/~fielding/pubs/dissertation/top.htm>. Acesso em: 4 mar. 2026.

GAREY, M. R.; JOHNSON, D. S. Computers and Intractability: A Guide to the Theory of NP-Completeness. New York: W. H. Freeman, 1979.

GEEKSFORGEEEKS. Unified Modeling Language (UML) Class Diagrams. 2024. Disponível em: <https://www.geeksforgeeks.org/system-design/unified-modeling-language-uml-class-diagrams/>. Acesso em: 11 mar. 2026.

GLOVER, F. Future paths for integer programming and links to artificial intelligence. Computers & Operations Research, v. 13, n. 5, p. 533–549, 1986.

HARALICK, R. M.; ELLIOT, G. L. Increasing tree search efficiency for constraint satisfaction problems. Artificial Intelligence, v. 14, n. 3, p. 263–313, 1980.

JOHNSON, D. S. et al. Worst-case performance bounds for simple one-dimensional packing algorithms. SIAM Journal on Computing, v. 3, n. 4, p. 299–325, 1974.

KIRKPATRICK, S.; GELATT, C. D.; VECCHI, M. P. Optimization by simulated annealing. Science, v. 220, n. 4598, p. 671–680, 1983.

MÜLLER, T.; MURRAY, K. Comprehensive approach to student sectioning. Annals of Operations Research, v. 181, n. 1, p. 479–497, 2010.

MYERS, G. J.; SANDLER, C.; BADGETT, T. The Art of Software Testing. 3. ed. Hoboken: Wiley, 2011.

PRESSMAN, R. S.; MAXIM, B. R. Engenharia de Software: Uma Abordagem Profissional. 8. ed. Porto Alegre: McGraw-Hill, 2016.

RICHARDSON, L.; RUBY, S. RESTful Web Services. Sebastopol: O'Reilly Media, 2007.

RUMBAUGH, J. et al. Object-Oriented Modeling and Design. Englewood Cliffs: Prentice Hall, 1991.

RUSSELL, S.; NORVIG, P. Artificial Intelligence: A Modern Approach. 4. ed. Hoboken: Pearson, 2020.

SCHAERF, A. A survey of automated timetabling. Artificial Intelligence Review, v. 13, n. 2, p. 87–127, 1999.

SOMMERVILLE, I. Engenharia de Software. 9. ed. São Paulo: Pearson, 2011.

STONEBRAKER, M.; ROWE, L. A. The design of POSTGRES. ACM SIGMOD Record, v. 15, n. 2, p. 340–355, 1986.

WALLS, C. Spring Boot in Action. Shelter Island: Manning Publications, 2019.

WREN, A. Scheduling, timetabling and rostering: a special relationship? In: PRACTICE AND THEORY OF AUTOMATED TIMETABLING, 1., 1995, Edinburgh. Lecture Notes in Computer Science. Berlin: Springer, 1996. v. 1153, p. 46–75.

YOU, E. Vue.js Guide. 2014. Disponível em: <https://vuejs.org/guide>. Acesso em: 18 mar. 2026.